EXPERIMENT NO. 06 (Software) – Exploring Principles of Congestion in TCP



Name: Dibyanshu Shekhar Dey
Roll Number: 25EC64R23

Design and Simulation Lab: Communication Networks
Department of Electronics & Electrical Communication Engineering
Indian Institute of Technology Kharagpur

AIM:

- Part I: Baseline Network Exploration
- Part II: Congestion Control Design
- Part III: MultiFlow congestion control
- Part IV: (Challenge) From Congestion control to *Congestion Avoidance*

INTRODUCTION:
Congestion in network occurs when packet traffic exceeds link capacity, therefore causing packet loss and performance degradation. It is unavoidable, and therefore it is more easy to manage it using different and proper control mechanisms. A good congestion control must prevent overload, ensure fairness among the users, and utilize the bandwidth associated [1].

THEORY:
TCP congestion control helps in regulating the data transmission using a congestion window (cwnd) and feedback from the network to prevent overload. It consists of three phases: Slow Start (exponential growth), Congestion Avoidance (linear growth), and Congestion Detection (reducing rate and restarting or adjusting the phase) [1].

AIMD (Additive Increase Multiplicative decrease) is a feedback control algorithm that is used in TCP, where the congestion window (cwnd) grows linearly when no congestion is detected and decreases sharply when congestion occurs (hence multiplicative decrease)[2]. It combines additive increase and multiplicative decrease to regulate network traffic efficiently. The multiplicative decrease can be done by any factor (in most of the experiments / implementation it is taken as 2) [3].

Some other algorithms include AIAD, MIMD and MIAD. (M = Multiplicative, I = Increase, A = Additive, D = Decrease) [3].

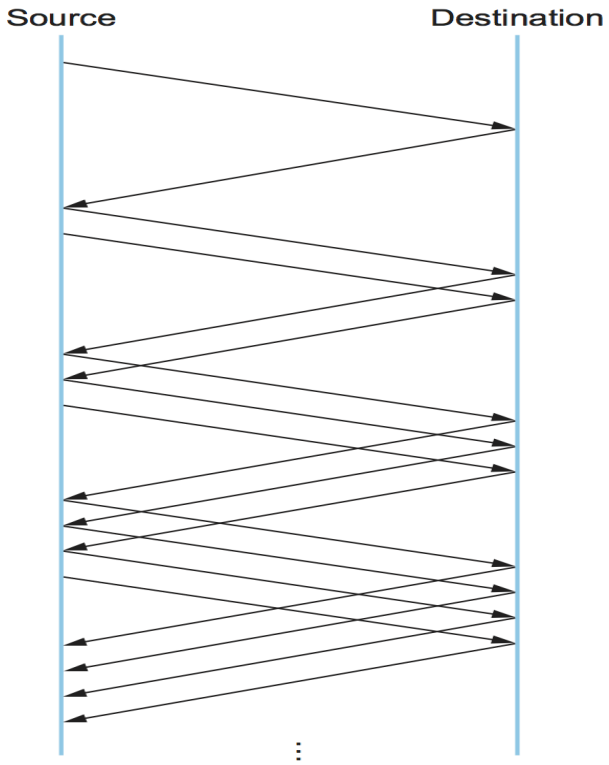


Figure 1: Packets in transit during additive increase, with one packet being added each RTT [4]

Algorithm 1 Additive Increase Multiplicative Decrease (AIMD)

Initialize $cwnd = 1$
Choose constant K
Choose constant K
if loss_event **then**
  $cwnd = \frac{cwnd}{K'}$
else
  $cwnd = cwnd + K$
end if

The value of K in the above algorithm can be chosen based on the system performance. Also the AIAD, MIMD, MIAD algorithms can be used by just changing the if else conditions.

RESULTS:

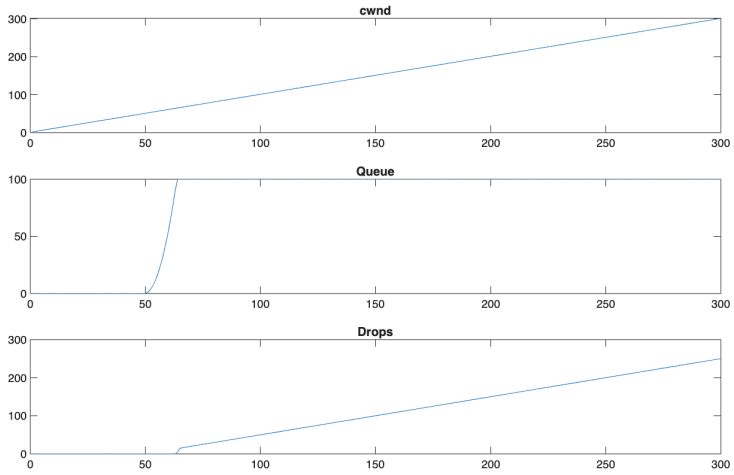


Figure 2: Part 1 Task A. The output of the code that was already in the lab report.

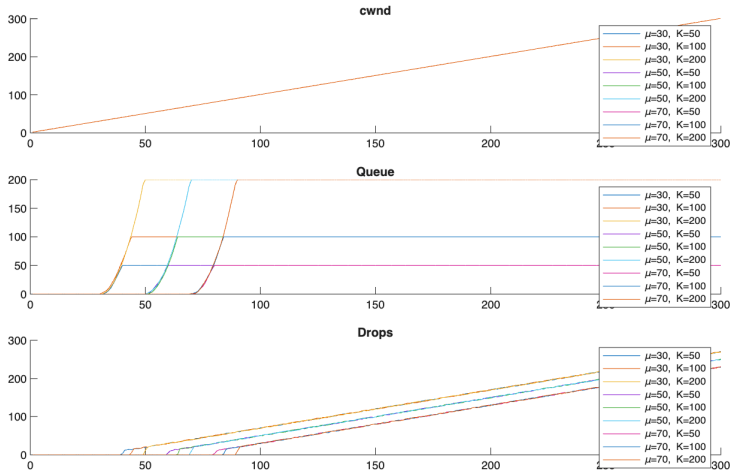


Figure 3: Part 1 Task B. Output based on different values of μ and K

Maximum and average of cwnd, queue and drops for different values of μ and K

```
The max of the cwnd for 30 and 50 is 301
The average of the queue for 30 and 50 is 44.0267
The max of the queue for 30 and 50 is 50
The average of the drops for 30 and 50 is 121.7467
The max of the drops for 30 and 50 is 269
The average of the cwnd for 30 and 100 is 151.5
The max of the cwnd for 30 and 100 is 301
The average of the queue for 30 and 100 is 87.1533
The max of the queue for 30 and 100 is 100
The average of the drops for 30 and 100 is 121.6533
The max of the drops for 30 and 100 is 270
The average of the cwnd for 30 and 200 is 151.5
The max of the cwnd for 30 and 200 is 301
The average of the queue for 30 and 200 is 171.93
The max of the queue for 30 and 200 is 200
The average of the drops for 30 and 200 is 121.3
```

```
The max of the drops for 30 and 200 is 270
The average of the cwnd for 50 and 50 is 151.5
The max of the cwnd for 50 and 50 is 301
The average of the queue for 50 and 50 is 40.81
The max of the queue for 50 and 50 is 50
The average of the drops for 50 and 50 is 104.4233
The max of the drops for 50 and 50 is 250
The average of the cwnd for 50 and 100 is 151.5
The max of the cwnd for 50 and 100 is 301
The average of the queue for 50 and 100 is 80.4033
The max of the queue for 50 and 100 is 100
The average of the drops for 50 and 100 is 104.2667
The max of the drops for 50 and 100 is 251
The average of the cwnd for 50 and 200 is 151.5
The max of the cwnd for 50 and 200 is 301
The average of the queue for 50 and 200 is 158.6767
The max of the queue for 50 and 200 is 200
The average of the drops for 50 and 200 is 103.9467
The max of the drops for 50 and 200 is 250
The average of the cwnd for 70 and 50 is 151.5
The max of the cwnd for 70 and 50 is 301
The average of the queue for 70 and 50 is 37.43
The max of the queue for 70 and 50 is 50
The average of the drops for 70 and 50 is 88.3767
The max of the drops for 70 and 50 is 230
The average of the cwnd for 70 and 100 is 151.5
The max of the cwnd for 70 and 100 is 301
The average of the queue for 70 and 100 is 73.76
The max of the queue for 70 and 100 is 100
The average of the drops for 70 and 100 is 88.18
The max of the drops for 70 and 100 is 230
The average of the cwnd for 70 and 200 is 151.5
The max of the cwnd for 70 and 200 is 301
The average of the queue for 70 and 200 is 145.1767
The max of the queue for 70 and 200 is 200
The average of the drops for 70 and 200 is 87.9067
The max of the drops for 70 and 200 is 230
```

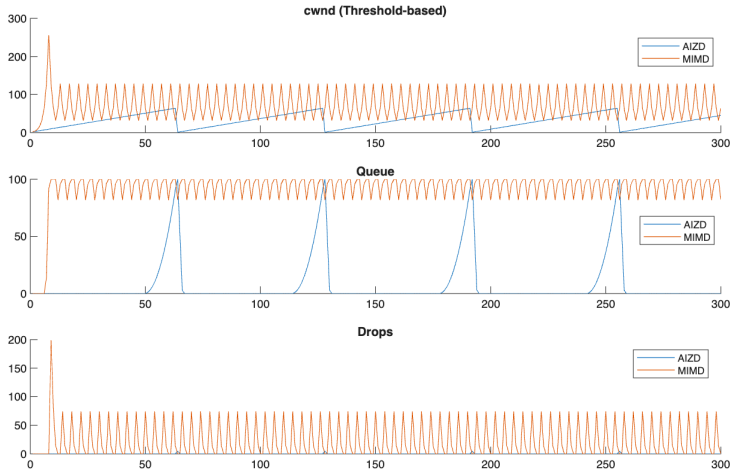


Figure 4: Part 1 Task C Additive increase Zero Decrease, multiplicative increase multiplicative decrease (done in class)

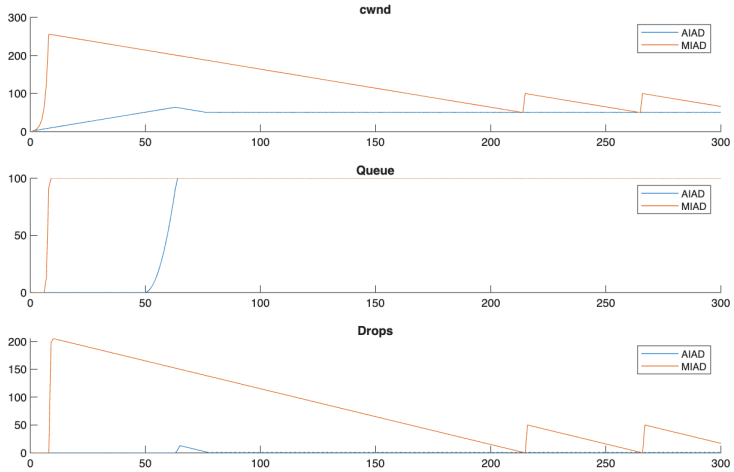


Figure 5: Part 2 Task D (a) Additive increase Additive decrease, Multiplicative increase, additive decrease

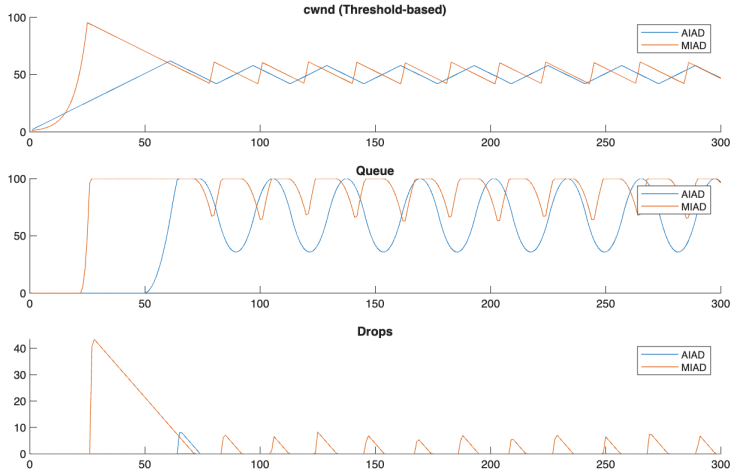


Figure 6: Part 2 Task D (b) Additive increase Additive decrease, Multiplicative increase, additive decrease with threshold introduced.

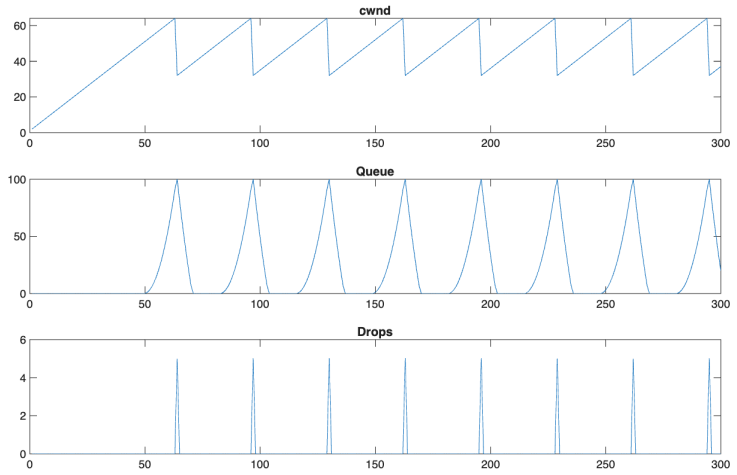


Figure 7: Part 2 Task E Additive increase multiplicative decrease

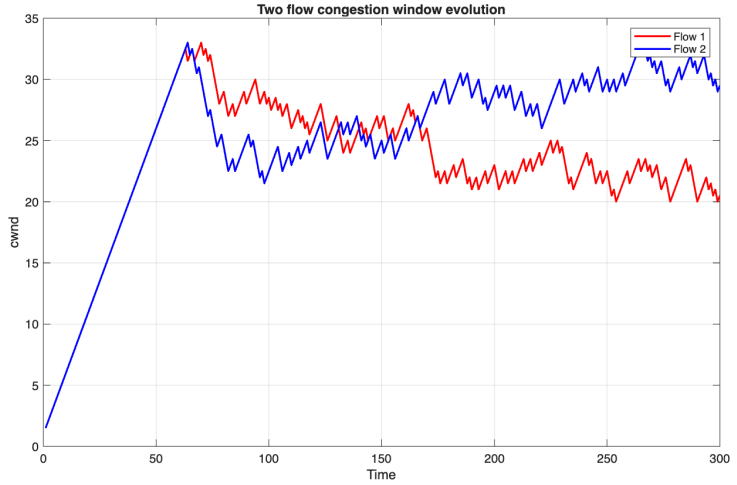


Figure 8: Part 3 Task F A - Two competing flow - done in the class/lab

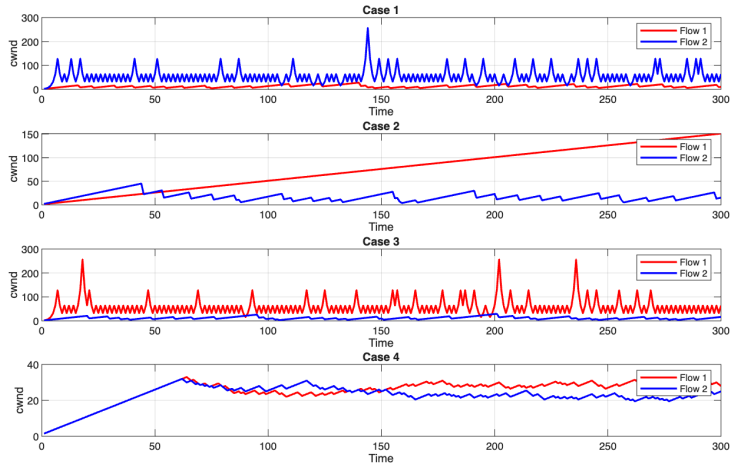


Figure 9: Part 3 Task F B Different schemes used for two competing flow.

```
Total packets drop:
   5.4935e+03
```

```
Jain fairness index:
    0.7203

Total packets drop:
   1.3436e+04

Jain fairness index:
    0.7099

Total packets drop:
   5.9228e+03

Jain fairness index:
    0.6950

Total packets drop:
  441.5000

Jain fairness index:
    0.9966
```

DISCUSSION:

Part 1

Does the queue stabilize or keep growing? What does stability mean to you?

From the plots, it can be seen that the queue initially grows rapidly, then hits the buffer limit K and then stays there. After that, it remains constant (flat line). So the queue stabilizes at K, due to buffer limit. A system is stable if the queue remains bounded and does not grow indefinitely.

How does cwnd growth relate to queue behavior?

cwnd increases linearly (which is an uncontrolled growth). Queue is initially small, then grows rapidly and hits limit K. As cwnd increases, arrivals increase. When arrival rate is greater than service rate μ, queue starts growing rapidly, and once full, extra packerts start dropping.

Is increasing the sending rate always beneficial?

Increasing sending rate is only beneficial up to the link capacity μ. Past that, it leads to congestion, packet delay and loss.

How do random arrivals affect stability? Comment clearly.

Random arrivals make the queue and drops fluctuate instead of following smooth, predictable patterns (see the graph of the results Part 1 Task B). These small randomness causes perturbations, while high randomness can lead to sudden congestion spikes and instability. Overall, randomness makes the system more realistic but less predictable, sometimes delaying or abruptly triggering congestion.

Part 2

Do you observe oscillatory behavior in cwnd?

Yes.

Why does cwnd increase slowly but decrease sharply?
cwnd increase slowly due to linear increase (additive) behavior, whereas the sharp decrease is due to multiplicative decrease, which reduces its value by half.

How does the queue behave now?
The queue no longer stays saturated and instead oscillates around the threshold K. It follows a periodic pattern of linear increase, drop, and then it repeats. So overall, the system is dynamically stable with no persistent overflow and only intermittent drops.

Part 3
Why does TCP exhibit sawtooth behavior?
This sawtooth behavior is due to Additive increase, multiplicative decrease procedure that has been used. Additive increase causes the graph to go linearly, where multiplicative decrease causes the graph to drop fast.

What role does packet loss play in stability and fairness?
Packet loss acts as a feedback for congestion i.e. it forces flows to reduce their sending rate, preventing overload which improves stability.

How do different backoff rules impact queue occupancy?
AIMD leads to smooth, stable behavior with the queue staying close to capacity without large fluctuations. MIMD, being more aggressive, causes sharp changes and significant fluctuations in the queue.

Future Work: Part D.
ACKNOWLEDGEMENT:
I would like to share my thanks to Prof. Amitalok J Budkuley for giving an introudtion to congestion control in TCP and engaging in an interactive session for this lab. I would like to thank my colleagues Mayank Singh, Saptarshi Bhattacharjee and Soumyajit Adak for collaborating in lab throughout the experiment.

References

[1] Educative, *What is congestion control in tcp?* `https://www.educative.io/answers/what-is-congestion-control-in-tcp`.

[2] GeeksforGeeks, *Tcp congestion control*, `https://www.geeksforgeeks.org/computer-networks/tcp-congestion-control/`.

[3] GeeksforGeeks, *Aimd algorithm*, `https://www.geeksforgeeks.org/computer-networks/aimd-algorithm/`.

[4] L. L. Peterson and B. S. Davie, *Computer Networks: A Systems Approach*, 5th ed. Morgan Kaufmann, 2011.

APPENDIX (CODE):

Part 1
`task_a.m`

```
% TCP CONGESTION CONTROL LAB     STARTER CODE

clear; clc; close all;

T = 300;
mu = 50;
K = 100;

cwnd = 1;
q = 0;

cwnd_trace = zeros(1,T);
q_trace = zeros(1,T);
drop_trace = zeros(1,T);

for t = 1:T

    arrivals = max(cwnd,0);

    q = q + arrivals;

    served = min(q, mu);
    q = q - served;

    if q > K
        drops = q - K;
        q = K;
    else
        drops = 0;
    end

    loss_event = (drops > 0);

    % BASELINE (no congestion control)
    cwnd = cwnd + 1;

    cwnd = max(cwnd,1);
    cwnd_trace(t) = cwnd;
    q_trace(t) = q;
    drop_trace(t) = drops;

end

figure;
subplot(3,1,1); plot(cwnd_trace); title('cwnd');
subplot(3,1,2); plot(q_trace); title('Queue');
subplot(3,1,3); plot(drop_trace); title('Drops');
```

Part 1

task_b.m

```
% TCP CONGESTION CONTROL LAB     STARTER CODE
% Vary    and K (e.g.,    = 30, 50, 70, K = 50, 100, 200).
% Observe changes in queue and drops.
% Introduce slight randomness in arrivals:
% arrivals = max(cwnd + randi([  1  ,1]),0);
% Record:
% Maximum and average queue
```

```
% Maximum and average drops     Maximum and average cwnd

clear; clc; close all;

T = 300;
mu_vals = [30, 50, 70];
K_vals = [50, 100, 200];

figure;

for i = 1:length(mu_vals)
    for j = 1:length(K_vals)
        mu = mu_vals(i);
        K  = K_vals(j);

        cwnd = 1;
        q = 0;

        cwnd_trace = zeros(1,T);
        q_trace = zeros(1,T);
        drop_trace = zeros(1,T);

        for t = 1:T
            arrivals = max(cwnd + randi([-1, 1]), 0);

            q = q + arrivals;

            served = min(q, mu);
            q = q - served;

            if q > K
                drops = q - K;
                q = K;
            else
                drops = 0;
            end

            loss_event = (drops > 0);

            % BASELINE (no congestion control)
            cwnd = cwnd + 1;

            cwnd = max(cwnd,1);
            cwnd_trace(t) = cwnd;
            q_trace(t) = q;
            drop_trace(t) = drops;

        end
        % Plotting
        subplot(3,1,1);
        hold on;
        plot(cwnd_trace, 'DisplayName', ['\mu=' num2str(mu) ', K=' num2str(
            K)]);
        avg_cwnd = mean(cwnd_trace);
        %disp('The average of the cwnd for ' + num2str(mu) + ' and ' +
            num2str(K) + ' is ' + num2str(avg_cwnd));
        disp("The average of the cwnd for " + mu + " and " + K + " is " +
            avg_cwnd);
```

```
        max_cwnd = max(cwnd_trace);
        disp("The max of the cwnd for " + mu + " and " + K + " is " +
            max_cwnd);

        subplot(3,1,2);
        hold on;
        plot(q_trace, 'DisplayName', ['\mu=' num2str(mu) ',␣K=' num2str(K)
            ]);
        avg_queue = mean(q_trace);
        disp("The average of the queue for " + mu + " and " + K + " is " +
            avg_queue);
        max_queue = max(q_trace);
        disp("The max of the queue for " + mu + " and " + K + " is " +
            max_queue);

        subplot(3,1,3);
        hold on;
        plot(drop_trace, 'DisplayName', ['\mu=' num2str(mu) ',␣K=' num2str(
            K)]);
        avg_drop = mean(drop_trace);
        disp("The average of the drops for " + mu + " and " + K + " is " +
            avg_drop);
        max_drop = max(drop_trace);
        disp("The max of the drops for " + mu + " and " + K + " is " +
            max_drop);
    end
end

% title and legends
subplot(3,1,1);title('cwnd');legend;
subplot(3,1,2);title('Queue');legend;
subplot(3,1,3);title('Drops');legend;
```

Part 2

task_c.m

```
% TCP CONGESTION CONTROL LAB     STARTER CODE

clear; clc; close all;

T = 300;
mu = 50;
K = 100;

algorithms = {'AIZD', 'MIMD'};




for a = 1:length(algorithms)

    algo = algorithms{a};
    cwnd = 1;
    q = 0;


    cwnd_trace = zeros(1,T);
```

```
    q_trace = zeros(1,T);
    drop_trace = zeros(1,T);

    for t = 1:T

        arrivals = max(cwnd,0);

        q = q + arrivals;

        served = min(q, mu);
        q = q - served;

        if q > K
            drops = q - K;
            q = K;
        else
            drops = 0;
        end

        loss_event = (drops > 0);

        % Congestion control
        switch algo
                % task
                case 'AIZD'
                    if loss_event
                        cwnd = 0;
                    else
                        cwnd = cwnd + 1;
                    end

                % task
                case 'MIMD'
                    if loss_event
                        cwnd = cwnd /2;
                    else
                        cwnd = cwnd * 2;
                    end

         end

        cwnd = max(cwnd,1);
        cwnd_trace(t) = cwnd;
        q_trace(t) = q;
        drop_trace(t) = drops;


    end
    %  Plotting
    subplot(3,1,1);
    hold on;
    plot(cwnd_trace, 'DisplayName', algo);

    subplot(3,1,2);
    hold on;
    plot(q_trace, 'DisplayName', algo);

    subplot(3,1,3);
```

```
    hold on;
    plot(drop_trace, 'DisplayName', algo);

end

%  Final Plot Formatting
subplot(3,1,1);
title('cwnd (Threshold-based)');
legend;

subplot(3,1,2);
title('Queue');
legend;

subplot(3,1,3);
title('Drops');
legend;
```

Part 2

task_d_a.m

```
% TCP CONGESTION CONTROL     Task D
% clear; clc; close all;

T = 300;
mu = 50;
K = 100;

algorithms = {'AIAD', 'MIAD'};

figure;

for a = 1:length(algorithms)

    algo = algorithms{a};

    cwnd = 1;
    q = 0;

    cwnd_trace = zeros(1,T);
    q_trace = zeros(1,T);
    drop_trace = zeros(1,T);

    for t = 1:T

        arrivals = max(cwnd,0);
        q = q + arrivals;

        served = min(q, mu);
        q = q - served;

        if q > K
            drops = q - K;
            q = K;
        else
            drops = 0;
        end

```

```
        loss_event = (drops > 0);

        %  Different Algorithms
        switch algo
            % task
            case 'AIAD'
                if loss_event
                    cwnd = cwnd - 1;
                else
                    cwnd = cwnd + 1;
                end

            % task
            case 'MIAD'
                if loss_event
                    cwnd = cwnd - 1;
                else
                    cwnd = cwnd * 2;
                end

        end

        cwnd = max(cwnd,1);
        cwnd_trace(t) = cwnd;
        q_trace(t) = q;
        drop_trace(t) = drops;

    end
    %  Plotting
    subplot(3,1,1);
    hold on;
    plot(cwnd_trace, 'DisplayName', algo);

    subplot(3,1,2);
    hold on;
    plot(q_trace, 'DisplayName', algo);

    subplot(3,1,3);
    hold on;
    plot(drop_trace, 'DisplayName', algo);

end

% Plot Formatting
subplot(3,1,1);
title('cwnd');
legend;

subplot(3,1,2);
title('Queue');
legend;

subplot(3,1,3);
title('Drops');
legend;
```

Part 2

task_d_b.m

```
% TCP CONGESTION CONTROL     THRESHOLD-BASED (ALL ALGORITHMS)

clear; clc; close all;

T = 300;
mu = 50;
K = 100;

q_thresh = 0.7 * K;   % threshold (70% of buffer)

algorithms = {'AIAD', 'MIAD'};

figure;

for a = 1:length(algorithms)

    algo = algorithms{a};

    cwnd = 1;
    q = 0;

    cwnd_trace = zeros(1,T);
    q_trace = zeros(1,T);
    drop_trace = zeros(1,T);

    for t = 1:T

        arrivals = max(cwnd,0);
        q = q + arrivals;

        served = min(q, mu);
        q = q - served;

        if q > K
            drops = q - K;
            q = K;
        else
            drops = 0;
        end

        %  Threshold-based event
        thresh_event = (q > q_thresh);

        %  Algorithms using threshold
        switch algo

            case 'AIAD'
                if thresh_event
                    cwnd = cwnd - 1;
                else
                    cwnd = cwnd + 1;
                end

            case 'MIAD'
                if thresh_event
                    cwnd = cwnd - 1;
                else
```

```
                    cwnd = cwnd * 1.2;
                end

        end

        cwnd = max(cwnd,1);

        cwnd_trace(t) = cwnd;
        q_trace(t) = q;
        drop_trace(t) = drops;

    end

    %  Plotting
    subplot(3,1,1);
    hold on;
    plot(cwnd_trace, 'DisplayName', algo);

    subplot(3,1,2);
    hold on;
    plot(q_trace, 'DisplayName', algo);

    subplot(3,1,3);
    hold on;
    plot(drop_trace, 'DisplayName', algo);

end

%  Final Plot Formatting
subplot(3,1,1);
title('cwnd (Threshold-based)');
legend;

subplot(3,1,2);
title('Queue');
legend;

subplot(3,1,3);
title('Drops');
legend;
```

Part 2

task_e.m

```
% TCP CONGESTION CONTROL LAB     STARTER CODE

clear; clc; close all;

T = 300;
mu = 50;
K = 100;

cwnd = 1;
q = 0;

cwnd_trace = zeros(1,T);
q_trace = zeros(1,T);
drop_trace = zeros(1,T);
```

```
15
16  for t = 1:T
17
18      arrivals = max(cwnd,0);
19
20      q = q + arrivals;
21
22      served = min(q, mu);
23      q = q - served;
24
25      if q > K
26          drops = q - K;
27          q = K;
28      else
29          drops = 0;
30      end
31
32      loss_event = (drops > 0);
33
34      % BASELINE AIMD
35      if loss_event
36          cwnd = cwnd / 2;
37      else
38          cwnd = cwnd + 1;
39      end
40      cwnd_trace(t) = cwnd;
41      q_trace(t) = q;
42      drop_trace(t) = drops;
43
44  end
45
46  figure;
47  subplot(3,1,1); plot(cwnd_trace); title('cwnd');
48  subplot(3,1,2); plot(q_trace); title('Queue');
49  subplot(3,1,3); plot(drop_trace); title('Drops');
```

Part 3

task_f_a.m

```
1   clear;
2   clc;
3   close all;
4
5   T = 300; % number of time steps
6   mu = 50; %  service rate
7   K = 100; % buffer size
8
9   %initialise congestion window
10  cwnd1 = 1;
11  cwnd2 = 1;
12  q = 0;  % queue size
13
14  cwnd1_trace = zeros(1, T);
15  cwnd2_trace = zeros(1, T);
16  q_trace =  zeros(1, T);
17  drop_trace = zeros(1, T);
18
19  for t = 1:T
```

```
    arrivals1 = max(cwnd1,0);
    arrivals2 = max(cwnd2,0);
    total_arrivals = arrivals1 + arrivals2;

    q = q + total_arrivals;

    served = min(q, mu);
    q = q - served;

    if q > K
        drops = q - K; % packets dropped
        q = K;
    else
        drops = 0;
    end

    % example of proportional drops based on contribution
    if total_arrivals > 0
        p1 = arrivals1 / total_arrivals;
        p2 = arrivals2 / total_arrivals;
    else
        p1 = 0;
        p2 = 0;
    end

    loss_event1 = (drops > 0) && (rand < p1);
    loss_event2 = (drops > 0) && (rand < p2);

    % baseline, no congestion control
    if loss_event1
        cwnd1 = max(1, cwnd1 - 1);
    else
        cwnd1 = cwnd1 + 0.5;
    end

    if loss_event2
        cwnd2 = max(1, cwnd2 - 1);
    else
        cwnd2 = cwnd2 + 0.5;
    end

    cwnd1_trace(t) = cwnd1;
    cwnd2_trace(t) = cwnd2;
    q_trace(t) = q;
    drop_trace(t) = drops;

end

figure;
plot(cwnd1_trace, 'r', 'LineWidth', 1.5); hold on ;
plot(cwnd2_trace, 'b', 'LineWidth', 1.5);
xlabel('Time');
ylabel('cwnd');
title('Two flow congestion window evolution');
legend('Flow 1', 'Flow 2');
grid on;
disp("Total packets drop: ");
disp(sum(drop_trace));
```

```
x1 = mean(cwnd1_trace);
x2 = mean(cwnd2_trace);
J = ((x1 + x2)^2)/(2*(x1^2 + x2^2));
disp("Jain fairness index: ");
disp(J);
```

Part 3

task_f_b.m

```
clear;
clc;
close all;

T = 300; % number of time steps
mu = 50; % service rate
K = 100; % buffer size

figure;

for case_id = 1:4

    % initialise congestion window
    cwnd1 = 1;
    cwnd2 = 1;
    q = 0;

    cwnd1_trace = zeros(1, T);
    cwnd2_trace = zeros(1, T);
    q_trace =  zeros(1, T);
    drop_trace = zeros(1, T);

    for t = 1:T

        arrivals1 = max(cwnd1,0);
        arrivals2 = max(cwnd2,0);
        total_arrivals = arrivals1 + arrivals2;

        q = q + total_arrivals;

        served = min(q, mu);
        q = q - served;

        if q > K
            drops = q - K;
            q = K;
        else
            drops = 0;
        end

        % proportional drops
        if total_arrivals > 0
            p1 = arrivals1 / total_arrivals;
            p2 = arrivals2 / total_arrivals;
        else
            p1 = 0;
            p2 = 0;
        end

```

```
        loss_event1 = (drops > 0) && (rand < p1);
        loss_event2 = (drops > 0) && (rand < p2);

        %  SWITCH CASE
        switch case_id

            % Case 1: cwnd1 = AIMD, cwnd2 = MIMD
            case 1
                % cwnd1 AIMD
                if loss_event1
                    cwnd1 = cwnd1 / 2;
                else
                    cwnd1 = cwnd1 + 1;
                end

                % cwnd2 MIMD
                if loss_event2
                    cwnd2 = cwnd2 / 2;
                else
                    cwnd2 = cwnd2 * 2;
                end

            % Case 2: cwnd1 AI (no decrease), cwnd2 AIMD
            case 2
                % cwnd1 additive increase only
                cwnd1 = cwnd1 + 0.5;

                % cwnd2 AIMD
                if loss_event2
                    cwnd2 = cwnd2 / 2;
                else
                    cwnd2 = cwnd2 + 1;
                end

            % Case 3: cwnd1 MIMD, cwnd2 AIMD
            case 3
                % cwnd1 MIMD
                if loss_event1
                    cwnd1 = cwnd1 / 2;
                else
                    cwnd1 = cwnd1 * 2;
                end

                % cwnd2 AIMD
                if loss_event2
                    cwnd2 = cwnd2 / 2;
                else
                    cwnd2 = cwnd2 + 1;
                end
            % Case 4: example given in the report
            case 4
                if loss_event1
                    cwnd1 = max(1, cwnd1 - 1);
                else
                    cwnd1 = cwnd1 + 0.5;
                end

                if loss_event2
```

```
                    cwnd2 = max(1, cwnd2 - 1);
                else
                    cwnd2 = cwnd2 + 0.5;
                end
        end

        cwnd1 = max(1, cwnd1);
        cwnd2 = max(1, cwnd2);

        cwnd1_trace(t) = cwnd1;
        cwnd2_trace(t) = cwnd2;
        q_trace(t) = q;
        drop_trace(t) = drops;

    end

    %  PLOTTING
    subplot(4,1,case_id);
    plot(cwnd1_trace, 'r', 'LineWidth', 1.5); hold on;
    plot(cwnd2_trace, 'b', 'LineWidth', 1.5);

    xlabel('Time');
    ylabel('cwnd');
    title(['Case␣' num2str(case_id)]);
    legend('Flow␣1', 'Flow␣2');
    grid on;

    disp("Total packets drop: ");
    disp(sum(drop_trace));
    x1 = mean(cwnd1_trace);
    x2 = mean(cwnd2_trace);
    J = ((x1 + x2)^2)/(2*(x1^2 + x2^2));
    disp("Jain fairness index: ");
    disp(J);

end
```